

Laboratorium 0

Wstęp do Matlaba

1. CELE

Zapoznanie się z podstawowymi cechami środowiska Matlab i przykładami popularnych instrukcji.

UWAGA: Więcej ogólnych informacji na temat Matlaba znajduje się w pliku `Matlab_introduction.pdf` file (dostępny w Moodle'u).

2. MATLAB – KILKA OGÓLNYCH ZASAD I PRZYKŁADÓW

Do wykonania poniższych ćwiczeń zalecane jest utworzenie skryptu (np. o nazwie `laboratorium_1.m`) i zapisywanie w tym skrypcie całego tworzonego kodu (więcej w sekcji 2.7).

2.1. Generowanie i reprezentacja danych/zmiennych

WAŻNE UWAGI:

1. W Matlabie WSZYSTKIE zmienne (i struktury również) traktowane są jako tablice (wektory, macierze 2D, macierze 3D, itp.). Na przykład, zapis `x`, `x(1)` czy `x(1,1)` mają takie samo znaczenie (jeśli użyte we właściwym kontekście).
2. Dane/zmienne w Matlabie są 1-indeksowane (w przeciwieństwie do 0-indeksowania np. w C/C++). Tak więc „matlabowe” `x(1,2)` powinno być zapisane w C++ jako `x(0,1)`.

Przykład deklaracji (i nadania wartości) zmiennej rzeczywistej i zespolonej:

```
x = 2.1; y = 2.1 + 1i*3.3;
```

Przykład deklaracji wektora wierszowego (poziomego) `X` i wektora kolumnowego (pionowego) `Y`

```
X = [1 4 6]; Y = [1; 4; 6]; albo Y = X';
```

Przykład deklaracji (i nadania wartości) macierzy 2x3

```
Y = [1 4 6; 5 6 9];
```

Ponieważ indeksowanie w Matlabie zaczynamy od 1 (a nie od 0) w powyższej macierzy

```
Y(2,1) = 5.
```

Przykład generowania wektora reprezentującego wartości wyrażenia/funkcji (np. `cosinus`)

```
t = -pi:0.01:pi;  
x1 = cos(t);
```

Przykładowe przydatne funkcje: `real`, `imag`, `size`, `length`, operator dwukropka (`:`), `ones`, `zeros`, `rand`, `clear`, `whos`.

Inne przykłady definiowania tablic (i nadania im defaultowych wartości):

```
A = zeros(5,2); B = 75*ones(3,3,3);
```

Przydatne operacje i operatory: `sum`, `*`, `^`, `&&`, `||`, `&`, `|`, `<`, `>`, `==`, `~=`

Przydatne słowa kluczowe: `if`, `else`, `for`, `case`, `while`, `end`

2.2. Działania na macierzach

“Zwykłe” operacje arytmetyczne mogą często (choć nie zawsze i pod pewnymi warunkami) reprezentować działania na macierzach.

Jeśli X i Y są macierzami o tych samych rozmiarach, można je dodawać i odejmować oraz wykonywać mnożenie i dzielenie („z kropką”) odpowiednio tych samych elementów obu macierzy.

$$X+Y, X-Y, X.*Y, X./Y$$

Do „zwykłego” mnożenia macierzy $X * Y$ wymagane jest, żeby liczba kolumn macierzy X oraz liczba wierszy macierzy Y były sobie równe.

2.3. Grafika

W przypadku kreślenia (wizualizacji) danych 1D, np. szeregów czasowych, postępujemy zazwyczaj w następujący (lub podobny) sposób:

```
figure(<numer>); plot(t, x1);  
title('wykres'), xlabel('czas'), ylabel('cosinus').
```

Inne przydatne funkcje: `stem`, `bar`, `grid`, `legend`, `hold`, `close all`.

2.4. Dane obrazowe (tworzenie, odczyt i zapis)

Matlab akceptuje (w sensie czytania i zapisu) obrazy w bardzo wielu formatach (`jpg`, `gif`, `bmp`, `png`, itd.) ale ich wewnętrzna reprezentacja jest zawsze taka sama:

- 2D macierze $M \times N$ w przypadku obrazów w poziomach szarości;
- 3D macierze $M \times N \times 3$ w przypadku obrazów kolorowych.

Wartości tych macierzy są liczbami z zakresu $0 \div 255$ (1-bajtowa liczba bez znaku) tzn. są to macierze typu `uint8`.

Przykład utworzenia „czarnego” obrazu kolorowego i monochromatycznego:

```
Im = uint8(zeros(M, N, 3)); Im = uint8(zeros(M, N));
```

Typowa sekwencja wczytania, przetwarzania, wyświetlania i zapisu obrazu:

```
Im = imread('obraz.bmp');  
imshow(Im); % wyświetlenie wczytanego obrazu  
ImZ = double(Im); % zamiana na postać zmiennoprzecinkową  
ImZ = przetwarzanie(ImZ);  
imshow(ImZ/255); % dlaczego dzielimy przez 255?  
ImNew = uint8(ImZ); % powrót do reprezentacji bajtowej  
imwrite(ImNew, 'obraz_nowy.jpg'); % zapis do foldera
```

2.5. Animacje (tworzenie, odczyt i zapis)

Matlab akceptuje (w sensie czytania i zapisu) pliki video w bardzo wielu formatach (`avi`, `mp3`, `wav`, , itd.). Pliki takie są odczytywane i zapisywane jako sekwencje indywidualnych obrazek (pojedynczych klatek). Typowa sekwencja odczytu (i zapisu) pliku video:

```
v = VideoReader('film_01.avi');  
v2 = VideoWriter('film_01.mp3', 'MPEG-4');  
open(v2);  
while hasFrame(v)  
    klatka = readFrame(v);  
    % ewentualne przetworzenie klatki  
    imresize(klatka, 0.5);  
    writeVideo(v2, klatka);  
end
```

```
close(v2);
```

2.6. Funkcje

Pliki matlabowe (M-files) mogą zawierać funkcje tworzone przez użytkownika. Tworząc plik zawierający tylko funkcję należy w pierwszej linii zdefiniować nazwę funkcji i jej argumenty, np:

```
function y = arraymean(x)
% oblicza skalarną średnią wszystkich elementów tablicy x
y = mean(x(:));
end
```

Nazwa pliku powinna odpowiadać nazwie funkcji (w tym przypadku **arraymean.m**).

Funkcje można również definiować lokalnie (w danym skrypcie). W takiej sytuacji należy zdefiniować je na końcu skryptu w Matlabie (ale na początku skryptu w Octave).

Takie podejście jest zalecane w tworzeniu dokumentacji laboratoryjnej (jeden skrypt zawierający wszystkie wywoływane w nim funkcje).

Uwaga:

Wyjaśnienia i przykłady dotyczące wybranych funkcji można uzyskać na kilka sposobów:

help <name_of_function> (na przykład **help size**) albo

doc <name_of_function>, albo

edit <name_of_function>

2.7. Dokumentacja

Instrukcja **publish** umożliwia łatwe i wygodne tworzenie dokumentacji (kod i wyniki graficzne) eksperymentów numerycznych realizowanych w Matlabie.

Ćwiczenie:** Opublikuj wyniki działania swojego skryptu (np. skryptu o nazwie **laboratorium_1.m** tworzonoego w trakcie zajęć na podstawie wzorca dostępnego w Moodle'u). W tym celu wykonaj instrukcję **publish('laboratorium_0.m', 'pdf')

Przeanalizuj treść otrzymanego pliku PDF.